

ENTITY FRAMEWORK

Основные команды LINQ

Where — фильтрация

Самая часто используемая команда. Она позволяет отобразить только те записи, которые соответствуют условию. Всё, что написано в Where, превращается в SQL-оператор WHERE и выполняется на сервере базы данных, а не в памяти приложения.

```
var shortSongs = db.Tracks.Where(t => t.Duration < 180).ToList();
```

Select — выборка определённых полей

По умолчанию EF возвращает все поля таблицы. Но иногда нужны не все данные, а только некоторые. Select позволяет спроектировать результат в нужную форму — либо анонимный объект, либо другой класс.

```
var songsNames = db.Tracks.Select(t => t.Title).ToList();
```

OrderBy — сортировка

Упорядочивает результат по возрастанию или убыванию.

```
var sortedTracks = db.Tracks.OrderBy(t => t.Duration).ToList(); // по возрастанию

var sortedTracksDesc = db.Tracks.OrderByDescending(t => t.Duration).ToList(); // по убыванию
```

Include — загрузка связей

По умолчанию EF не загружает связанные объекты — только данные из основной таблицы. Это сделано для производительности, чтобы не брать из базы лишнего. Если нужны связанные данные (например, товары вместе с их категориями), нужно явно указать это через Include.

```
var playlistsWithTracks = db.Playlists
    .Include(p => p.Tracks)
```

```
.ToList();
```

FirstOrDefault — получить один объект

Используется, когда нужна только первая запись, удовлетворяющая условию, или нужно найти запись по идентификатору. Возвращает либо найденный объект, либо null, если ничего не нашлось.

```
var shortTrack = db.Tracks.FirstOrDefault(t => t.Duration = 175);
```

CRUD – операции

```
using var db = new AppDbContext();

// Create
db.Users.Add(new User { Name = "Анна" });
db.SaveChanges();

// Read
var users = db.Users.ToList();

// Update
var user = db.Users.First();
user.Name = "Олег";
db.SaveChanges();

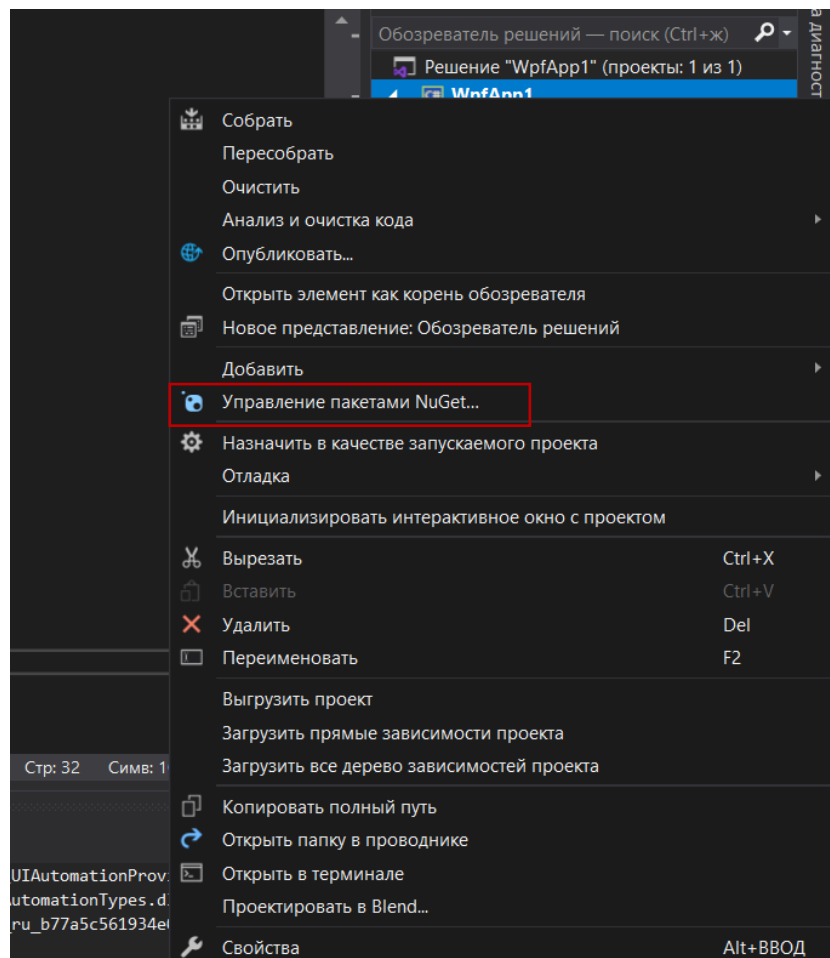
// Delete
db.Users.Remove(user);
db.SaveChanges();
```

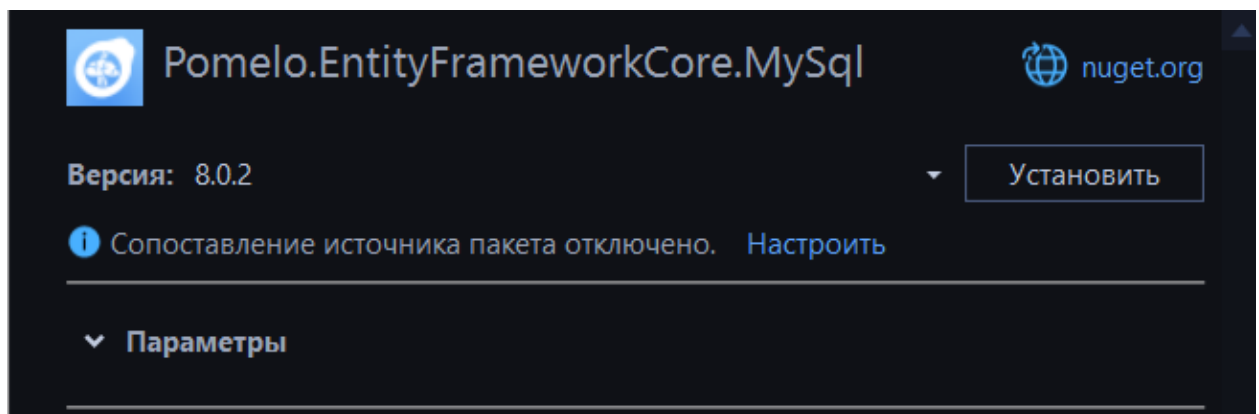
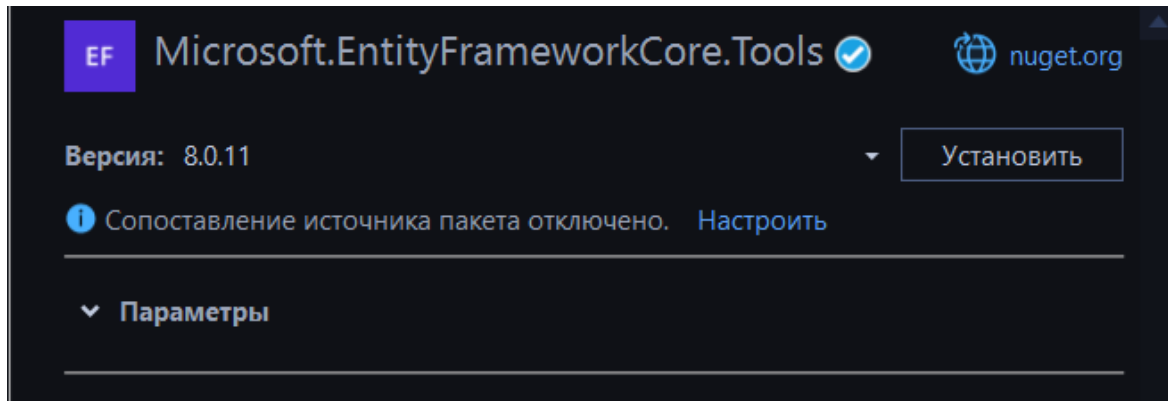
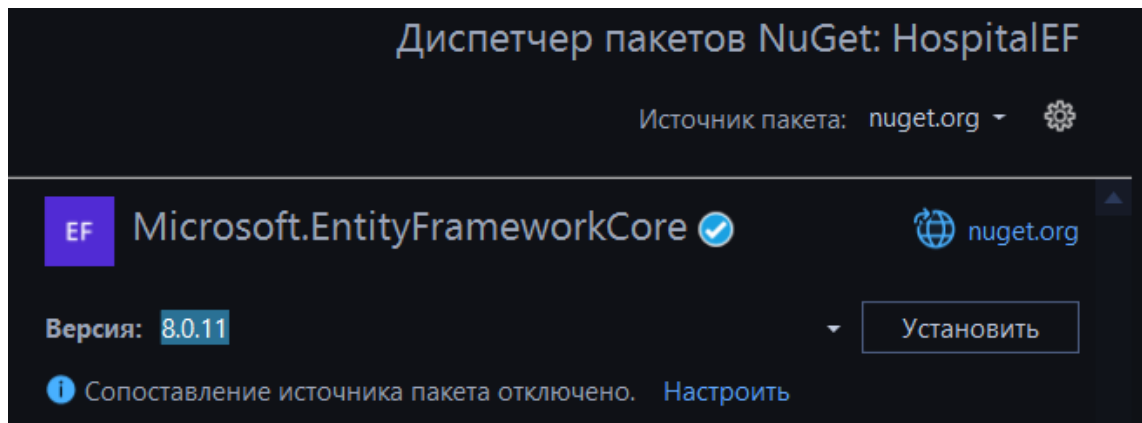
СОЗДАНИЕ ПРОЕКТА И ПОДКЛЮЧЕНИЕ ПАКЕТОВ

1. создайте консольное приложение, установите необходимые пакеты и подключите бд

2. Установите необходимые пакеты:

- Microsoft.EntityFrameworkCore (версия 8.0.11)
- Microsoft.EntityFrameworkCore.Tools (версия 8.0.11)
- Pomelo.EntityFrameworkCore.MySql (версия 8.0.2)





Обязательно устанавливайте версии, указанные на скриншотах, иначе работать не будет.

3. Для подключения созданной базы данных откройте PowerShell, перейдите в папку с файлом .csproj (cd Название папки (обычно так же как и название проекта)) и пропишите команду

```
dotnet ef dbcontext scaffold "строка соединения с базой данных"
Pomelo.EntityFrameworkCore.MySql --context-dir DbContexts --output-dir
Entities
```

Строка подключения выглядит следующим образом:

```
server=localhost; user=имя_пользователя(обычно root); password=пароль;  
database=название_бд
```

(порядок написания не важен)

Если после прописывания команды выходит ошибка:

Не удалось выполнить, поскольку указанная команда или файл не найдены. Возможные причины этого включают:

- * Вы допустили ошибку во встроенной команде dotnet.
- * Вы собирались выполнить программу .NET, но dotnet-ef не существует.
- * Вы собирались запустить глобальное средство, но в PATH не удалось найти исполняемый файл с префиксом dotnet и таким именем.

Необходимо прописать команду:

```
dotnet tool install --global dotnet-ef --version 8.0.11
```

А после прописать предыдущую команду еще раз

ЗАДАНИЯ

Нацелены на освоение и понимание работы с EF

Картинная галерея

Во время выполнения заданий вы будете использовать подход DataBase First

1. Создайте консольный проект, установите все необходимые библиотеки и подключите базу данных «ArtGalleryDB»
2. Получите список всех художников и выведите в консоль имя художника и страну, для вывода используйте цикл foreach
3. Получите список художников, родившихся после 1850 года. Результат вывести в консоль.
4. Получите список картин и отсортируйте их по убыванию. Выведите в консоль название картины и ее цену.
5. Получите список всех картин вместе с художниками. Выведите в консоль.
6. Добавьте нового художника в базу данных. Проверьте, что запись была добавлена.
7. Измените цену любой выбранной картины. Выведите в консоль старую и новую цены вместе с названием картины.
8. Удалите выставку по названию. Выведите все выставки до удаления и после.

9. Удалите выставку по названию.
10. Выведите название картины, год написания и имя художника.
11. Сгруппируйте картины по художникам. (используйте GroupBy)
12. Выведите список художников и среднюю цену их картин. (Average, GroupBy)
13. Пользователь вводит строку. Нужно найти все картины, где название содержит эту строку. (Contains)
14. Выведите 5 самых дорогих картин вместе с художниками.
15. Добавьте новую картину существующему художнику.
16. Получите список картин, которые дороже заданной цены (выбираете сами), написанных после заданного года, выведите вместе с художниками и отсортируйте по цене (по возрастанию)

Офис

1. Создайте WPF приложение, подключите все необходимые пакеты и подключите БД «OfficeDB».
2. В главном окне создайте Grid с двумя столбцами и двумя строками:
Первая строка — заголовок.
Вторая строка — основной контент.
Первый столбец — список сотрудников.
Второй столбец — информация о сотруднике.
3. В левую часть окна добавьте ListBox для отображения сотрудников.

В правую часть добавьте:

TextBlock — ФИО

TextBlock — Должность

TextBlock — Зарплата

4. Получите список сотрудников из базы данных. Сохраните данные в ObservableCollection.

5. Получите сотрудников вместе с отделами.

6. Создайте форму добавления сотрудника.

7. Добавьте:

TextBox — ФИО

TextBox — Должность

TextBox — Зарплата

8. Реализуйте изменение зарплаты выбранного сотрудника.

9. Получите сотрудников с зарплатой выше средней. Выведите результат в отдельный список.

10. Добавьте кнопку удаления сотрудника.